Business 41204

Machine Learning

Reinforcement Learning

Christian Hansen

University of Chicago
Booth School of Business

Damian Kozbur

University of Zurich
Economics

Outline:

1. Overview of Reinforcement Learning
2. Bandits
3. Playing Games
4. More Involved Examples

1. Overview of Reinforcement Learning

Reinforcement Learning

Reinforcement Learning (RL) – essentially learning by doing with rewards and penalties

Basic structure:

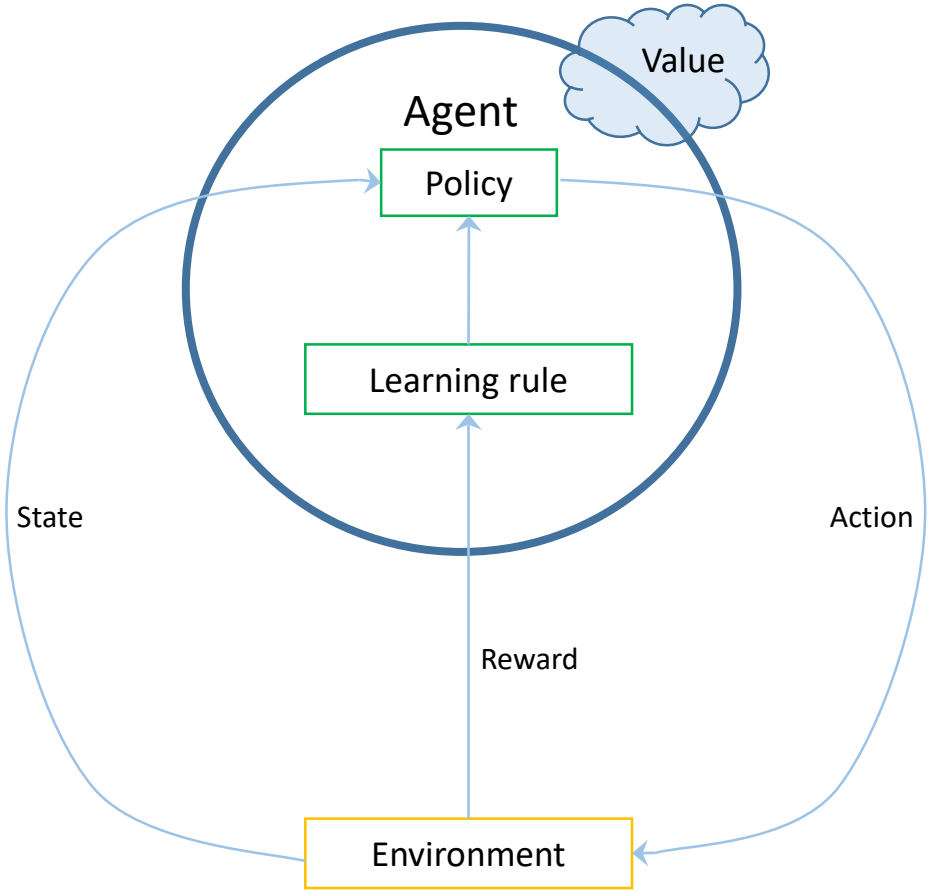
- Try different actions
  - *exploration*
- See outcomes from actions
- Become more likely to do what “works” and less likely to do what “fails”
  - *exploitation*
- Stylized version of how we (humans) learn lots of things

RL creates its own data via trial and error

- Need to be able to try many things many times
- Differs from supervised/unsupervised learning where data is given

Structure/Jargon

- **Agent:** learner/decision-maker trying to achieve some goal.
- **Environment:** context where agent operates
- **Action:** decision taken by agent.
- **State:** conditions under which agent takes actions at any given decision time. May change over time. May be influenced by actions. May have random/unknown elements.
- **Reward:** immediate feedback agent receives about action. May be positive or negative.
- **Policy:** rule that determines how agent behaves. Mapping between states and actions.
- **Value:** What we're really after. Basically the sum of rewards. Allows thinking about decisions that pay off at longer horizons.



A Little More Detail

One can view RL as trying to find a *policy function*, $\pi(s)$, that tells us what action, a, to take when the state is s

Define the *state value function for policy* π as

$$v_\pi(s) = E_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,|S_t = s\right]$$

- R_t is the reward at time t and γ is a discount factor
- Expected reward for following policy π when the state is s

Define the *action value function for policy* π as

$$q_\pi(s,a) = E_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,|S_t = s, A_t = a\right]$$

- Expected reward under policy π for taking action a when the state is s

Optimal policy then solves $\max_\pi v_\pi(s)$ AND $\max_\pi q_\pi(s,a)$

- Again, just trying to find policy that maximizes total/long term reward
- Let $v^*(s) = \max_\pi v_\pi(s)$ AND $q^*(s,a) = \max_\pi q_\pi(s,a)$
- Note that $\max_a q^*(s,a) = v^*(s)$

Bellman Equations

Unpacking things a bit more:

$$
\begin{aligned}
(i)\ q^*(s,a) &= \max_{\pi} E_{\pi}\left[R_t + \gamma \sum_{k=0}^{\infty} \gamma^k R_{t+k+2}\,|S_t = s, A_t = a\right] \\
&= E[R_t + \gamma v^*(S_{t+1})|S_t = s, A_t = a] \\
&= E[R_t + \gamma \max_{a'} q^*(S_{t+1}, a')|S_t = s, A_t = a]
\end{aligned}
$$

- The expected payoff from taking action a in state s now and then following the optimal policy is just the expected payoff from that decision today plus the expected payoff from following the optimal policy after that

$$
\begin{aligned}
(ii)\ v^*(s) &= \max_{a} q^*(s,a) \\
&= \max_{a} E[R_t + \gamma v^*(S_{t+1})|S_t = s, A_t = a]
\end{aligned}
$$

- The expected payoff of following the optimal policy at state s must equal the expected payoff of taking the best action from that state

(i) and (ii) are Bellman equations

- Solving or approximating these equations underlie most RL algorithms
- Chief intuition just that we don't think ONLY about rewards now but also expected payoffs from actions in the future

2. Bandits

Multi-armed bandits

Problem:

- n possible actions with unknown payoffs
- Need to repeatedly choose among these actions
  - Each time you choose, you get a random payoff depending on the action taken
  - Distribution generating rewards is not influenced by action
- Goal is to maximize (expected) total reward after some number of actions
- Solution is trivial with known payoff (distributions)
  - Always choose the one with highest (expected) payoff!
- Not quite "full" RL – actions don't influence states and no worry about short term actions impacting long term rewards

Examples:

- Have several potential advertisements you could run and want to choose the one that leads to the largest increase in purchases/revenue
- Have several candidate (experimental) treatments for a condition and want to choose the one that leads to the largest increase in well-being of patients

Exploit vs Explore

"Traditional Statistics" solution:

- run an experiment with many observations per potential treatment
  - Simple, robust. "Gold standard" for scientific discovery
  - BUT, inefficient – wastes resources learning about options that don't pay off
  - Focuses on *exploration*

"Greedy" solution:

- try each option a small number of times and then just stick with one that gave highest reward
  - Potentially beneficial in getting actual payoffs as "bad" policies are discarded
  - Great in a world with "stable" outcomes
  - BUT, prone to mistakes – does not spend resources *learning* to distinguish between different outcomes when the policy does not deliver "stable" outcomes
  - Focuses on *exploitation*

"Optimal" solution trades of exploration and exploitation

- Want to actually learn rewards of different choices without spending lots of time making bad decisions
- Under structure (likely unrealistic), can determine sophisticated strategies to produce theoretically best tradeoff between exploration/exploitation
- Not going to worry about this, but give ideas and talk about things that seem to work pretty well in practice

ε-Greedy Algorithms

Basic setup:

- At time *t+1*, know $\bar{X}_{j,t} = \frac{1}{N_{j,t}} \sum_{\tau=1}^{t} 1(arm\ j\ used\ at\ time\ \tau) x_{j,\tau}$ for $j = 1, \dots, n$
  - $x_{j,\tau}$ = payoff from trying action j at time τ
  - $N_{j,t} = \sum_{\tau=1}^{t} 1(arm\ j\ used\ at\ time\ \tau)$ = number of times action j tried up to time t
  - I.e. know average payoff up to time t
- Need to choose action for time $t + 1$

ε-Greedy Algorithm:

- Choose action with highest $\bar{X}_{j,t}$ with probability 1-ε
- Choose random action with probability ε

Idea:

- Forces all options to eventually be explored many times
- Spends a lot of time on what seems to be the best action
  - Can get stuck spending a lot time on “ok” actions

Upper Confidence Bound (UCB) Algorithm

Basic setup:

- At time *t+1*, know $\bar{X}_{j,t} = \frac{1}{N_{j,t}} \sum_{\tau=1}^{t} 1(arm\ j\ used\ at\ time\ \tau) x_{j,\tau}$ for $j = 1, \dots, n$
  - $x_{j,\tau}$ = payoff from trying action j at time τ
  - $N_{j,t} = \sum_{\tau=1}^{t} 1(arm\ j\ used\ at\ time\ \tau)$ = number of times action j tried up to time t
  - I.e. know average payoff up to time t
- Need to choose action for time $t + 1$

UCB Algorithm:

- Choose action with highest UCB Score:

$$UCB_j(t) = \bar{X}_{j,t} + \sqrt{\frac{2 \ln(t)}{N_{j,t}}}$$

  - Can be adapted to incorporate the sample variance as well

Idea:

- Optimistic and deterministic search algorithm
  - Prioritizes high reward $\bar{X}_{j,t}$
  - Encourages more exploration for options that have not been taken much (small $N_{j,t}$)
  - $\ln(t)$ terms encourages lots of exploration initially that slows over time (but because $\ln(t)$ is increasing and unbounded, always some "pressure" to explore)
    - $\ln(t)$ specifically because of concentration results (Hoeffding's inequality)

Code for Bandit Example

Bandit Illustration

Consider a stylized bandit example.

Need to choose among 10 options over a horizon of 1000 periods.

- At each period, reward for action j drawn from $N(\mu_j, 1)$

Look at the performance of 5 algorithms ("agents")

- Completely random
- ε-greedy with $\varepsilon = 0.1$
- Completely greedy
- UCB
- Thompson Sampling
  - A probabilistic algorithm based on Bayesian updating

Bandit Illustration

Looks like completely greedy is doing really well!

Do we think this generalizes beyond this one example?

[Code for Bandit Example]

Performance in one simulation:

	Completely Random	Epsilon-Greedy	Completely Greedy	UCB	Thompson Sampling
Average Reward	-0.13	1.54	1.76	1.66	1.69
Optimal Actions	108	892	988	913	938

Action 8 is the optimal action.

All algorithms learn the payoff to Action 8 pretty well.

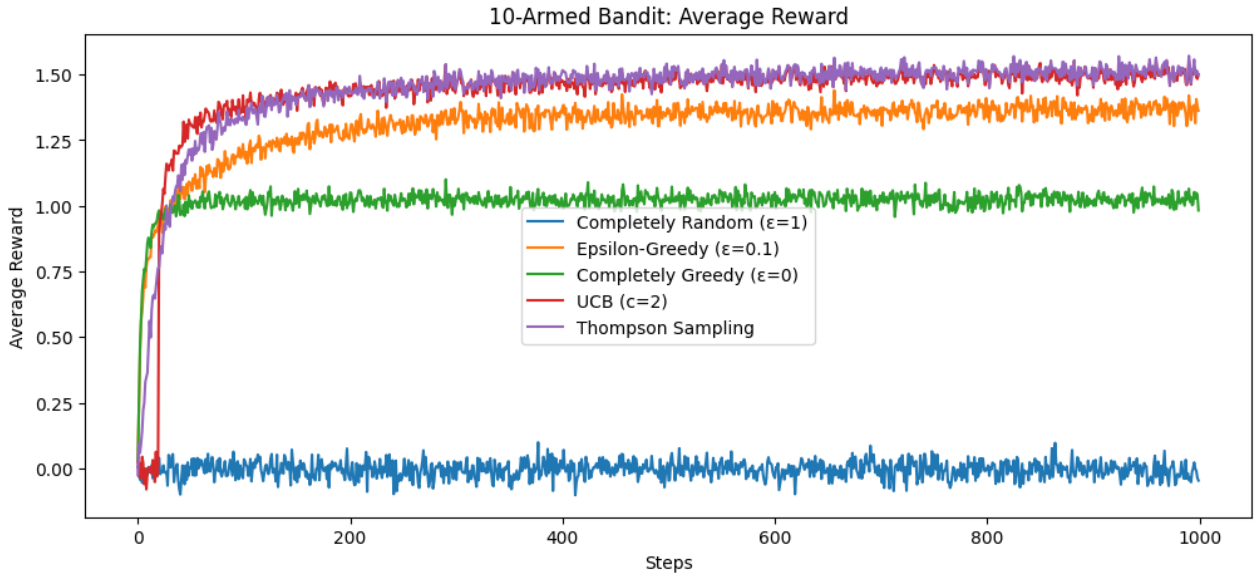
All algorithms learn Action 8 is much better than other actions.

What if we are interested in payoffs to other actions?

	True Reward	Completely Random	Epsilon-Greedy	Completely Greedy	UCB	Thompson Sampling
Action 1	0.23	0.18	0.40	-0.69	0.17	0.17
Action 2	1.03	1.18	0.89	-0.25	0.99	1.15
Action 3	-0.84	-0.93	-0.09	-0.44	-2.05	-0.86
Action 4	-0.59	-0.72	-0.60	-0.73	-1.56	-0.75
Action 5	-0.96	-1.00	-0.86	-0.44	-0.90	-0.68
Action 6	-0.22	-0.21	-0.46	-0.02	-0.87	0.14
Action 7	-0.62	-0.60	-0.89	-1.66	-0.57	-0.44
Action 8	1.84	1.93	1.77	1.78	1.78	1.78
Action 9	-2.05	-1.95	-2.20	0.00	-1.98	-1.13
Action 10	0.87	0.72	0.59	0.00	0.71	0.82

Bandit Illustration

[Code for Bandit Example]



Performance across 2000 simulations

- Each simulation uses different mean rewards for the actions

Figures show

- average (across 2000 simulations) reward at each time step
- fraction of instances (across 2000) where optimal action is selection at each time step

Algorithm	Average MSE (Optimal Action)	Average MSE (Across All Actions)
Completely Random	0.011	0.010
ε-Greedy	0.022	0.085
Completely Greedy	1.896	0.852
UCB	0.047	0.271
Thompson Sampling	0.009	0.130

Comments on Bandits

Things to think about:

- What would you do if you think the reward (distribution) changes over time?
- What do you think happens as you add more possible actions?
- What if action payoffs are similar?
  - What if action payoffs are highly variable?
- What if action payoffs depend on other features?
  - E.g. effectiveness of ad depends on how customer arrived at website
  - "contextual bandits"

- What is your goal?
  - Learn about all action payoffs as well as possible
  - Choose the best action as much as possible

3. Playing Games

Infinite Deck Blackjack

Let's look at somewhat more complicated example. We want to learn how to optimally play infinite deck blackjack

- Infinite decks means card counting is irrelevant
- Our goal is playing – i.e. trying to win – we are not going to worry about betting
  - Really simplifies our problem!

Problem Formulation:

- Agent: Player
- Actions: hit/stand (ignore double down, split, surrender)
- Environment: deck of cards, dealer's rules
- State variables: sum of player's cards (12-21: why?), whether player has a useable ace, dealer's card (1-10)
  - 200 total relevant states
- Rewards: Win (+1), Draw (0), Loss (-1)
  - Only receive reward when a game (*episode*) ends

Solving with Q-Learning

There are many algorithms for solving RL problems. Here, we'll present a simple one called Q-learning:

- Basic idea is trying to learn the *optimal* Q-value function:

$$Q(s, a) = E[R_t | S_t = s, A_t = a]$$

  - In words, the expected reward at time t when the state of the world is s for taking action a
  - If we had the *optimal* Q-value function (call it $Q^*(s, a)$), we'd know what to do in state s: Choose the action that maximizes $Q^*(s, a)$

Solving with Q-Learning

Q-learning "Algorithm"

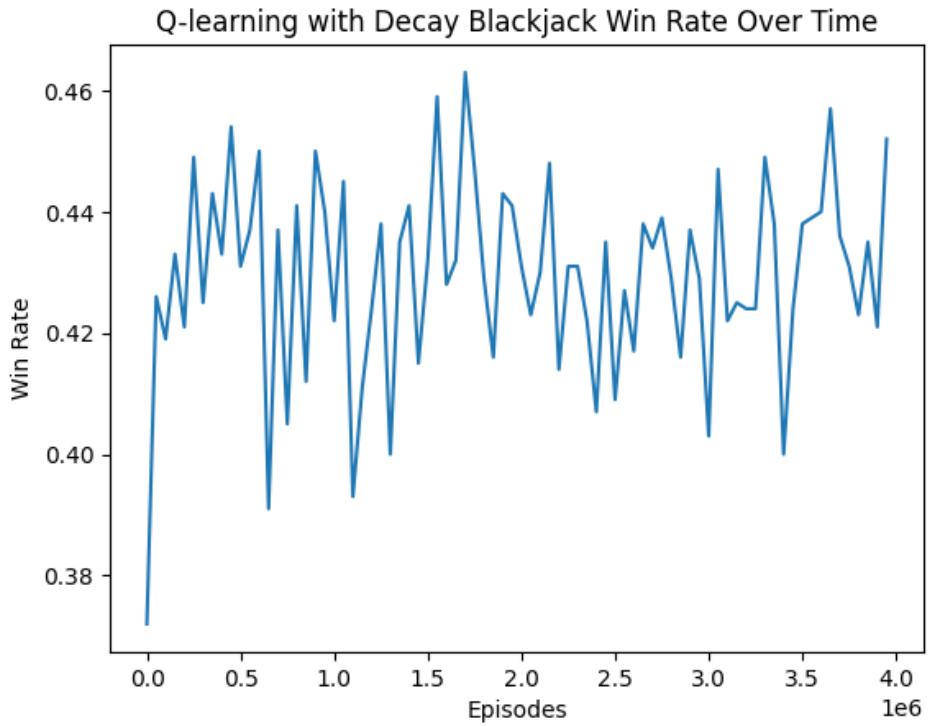
1. Make an initial guess for $Q(s, a)$
2. Repeat ***many*** times:
   1. Start a game (i.e. deal cards to player and dealer) and observe the initial state
   2. Use ε-greedy action choice:
      - Choose a random action with probability ε
      - Choose action $\arg\max_a \ Q_{cur}(s, a)$ where $Q_{cur}(s, a)$ is the current estimate of the Q-value function
   3. Perform action a, receive reward r, observe new state s'
   4. Update the Q-value function: $Q_{new}(s, a) = Q_{cur}(s, a) + \alpha \left[r + \gamma \max_{a'} Q_{cur}(s', a') - Q_{cur}(s, a) \right]$
      - α ("learning rate") and γ ("discount factor") are tuning parameters that control how quickly we update the Q function and how much we discount future rewards
      - Note this is essentially updating by approximating the Bellman equation
   5. Keep going until episode (game) ends and we observe win/loss/draw

Code for Blackjack Example

Blackjack Solution

Here, we run Q-learning for 4,000,000 episodes

- In this example, an episode is a hand of blackjack



Win rate of agent as it plays more hands.

Looks like it has learned to win 42-46% of hands (give or take)

Humans win around 42% (https://www.winstar.com/blog/blackjack-odds/)

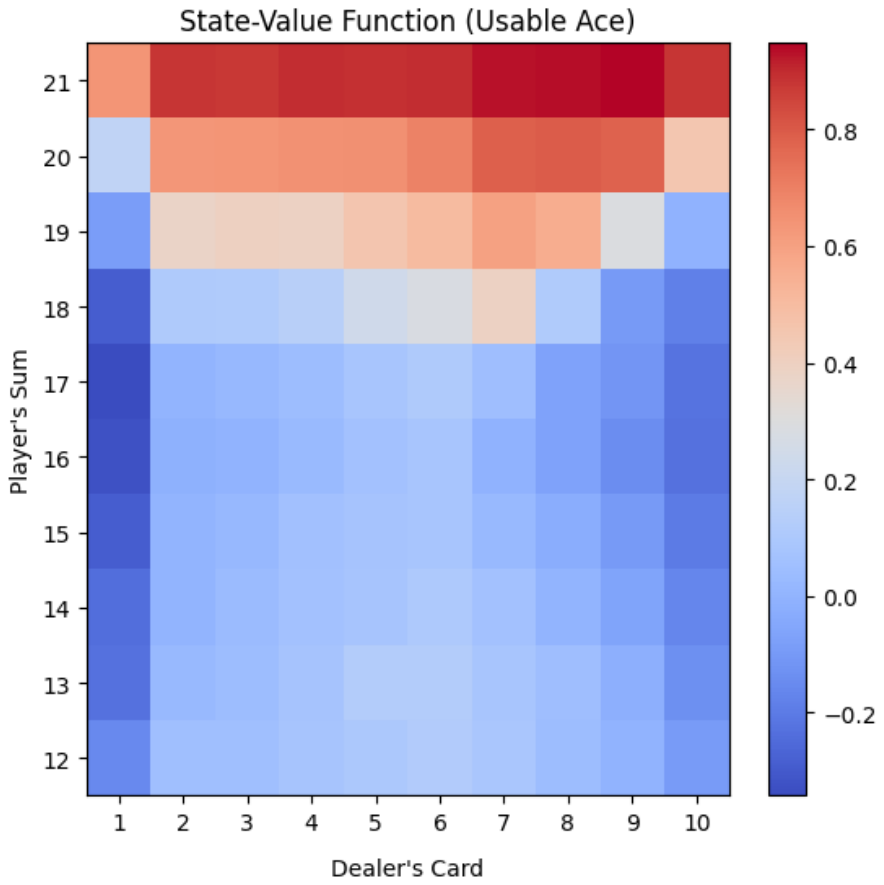
Blackjack State Value Function

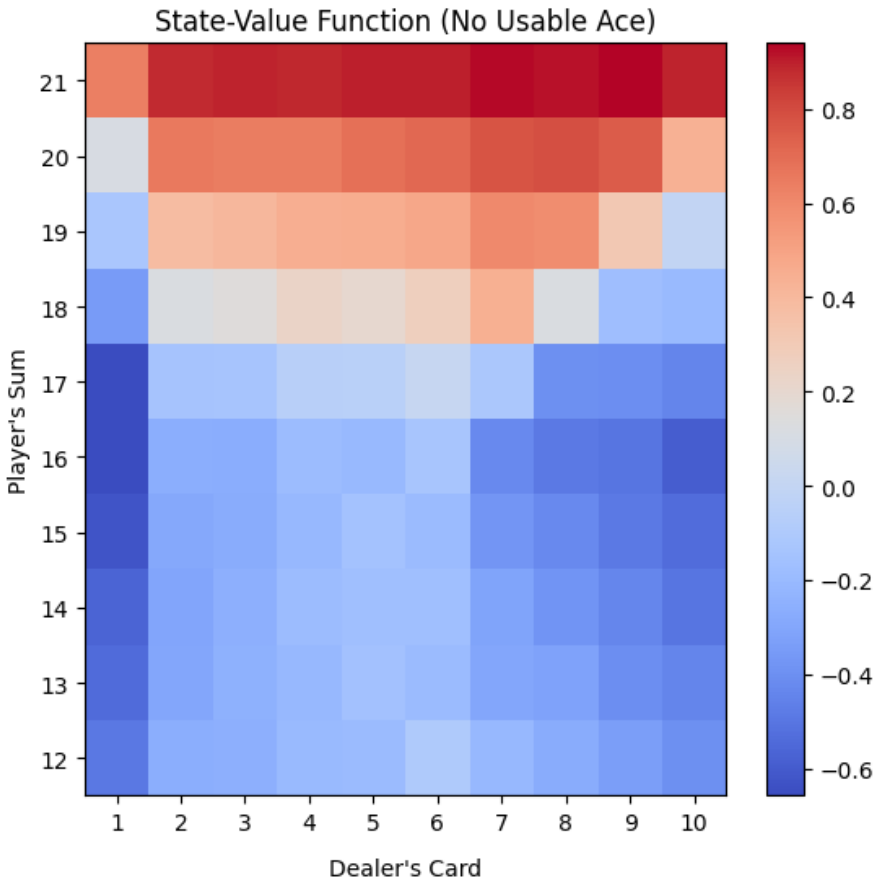
[Code for Blackjack Example]

Recall that we have defined our rewards as

- +1 (win)
- 0 (draw)
- -1 (lose)

The state value function gives you the expected reward from taking the optimal action in each state.





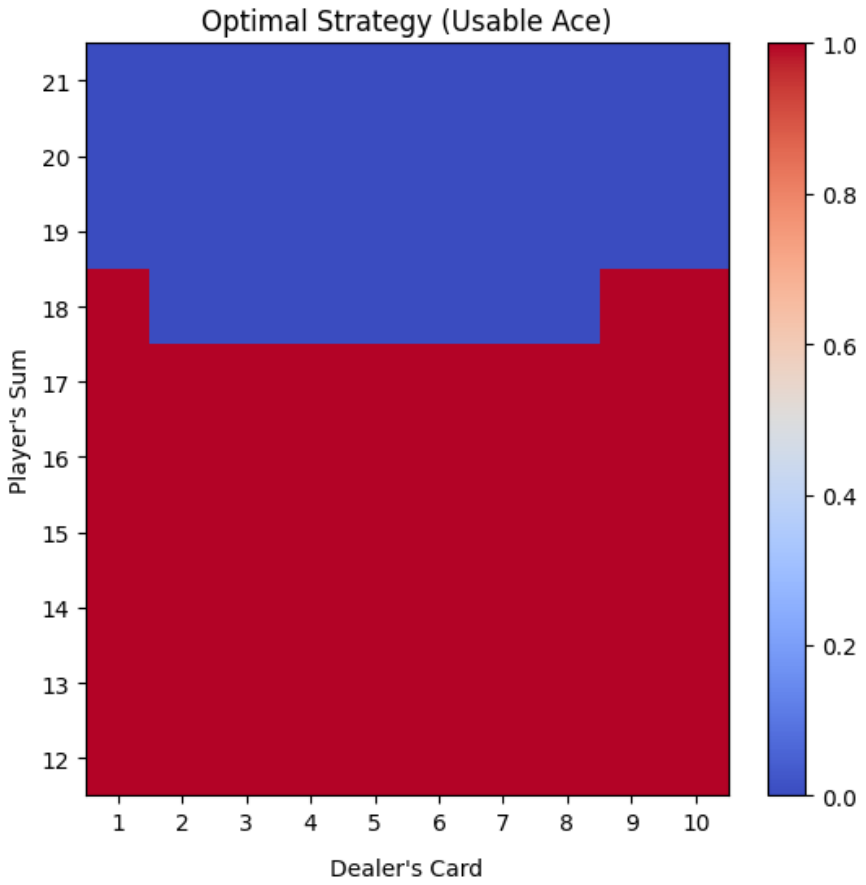
Blackjack Optimal Actions

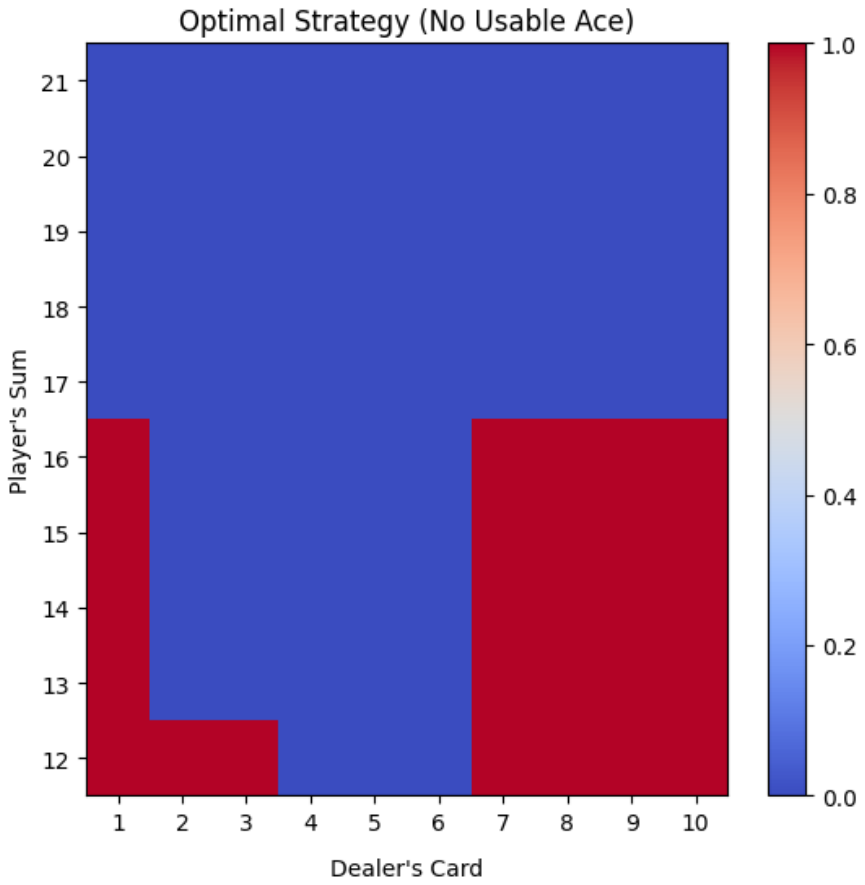
Code for Blackjack Example

Red denotes *hit*

Blue denotes *hold*

Almost identical to the “basic” strategy in Thorp (1966) Beat the Dealer





4. More Involved Examples

Large State Spaces

Let's think about single deck blackjack.

- With a single deck, card counting becomes relevant
  - To keep track of cards, need to enlarge the state space
  - Could imagine keeping track of all the cards left in the deck
  - State space is gigantic ≈ 840 million states
    - Could consider simpler counting strategies, but state space is still large
- Impractical to keep track of all states and hope to visit them many times!

RL with large input space proceeds by parameterizing action, value, and/or q-value functions

- E.g. specify them as linear models
- E.g. DeepRL – specify unknown functions as deep neural networks

Implementation typically requires

- Lots of computation
- Trial and error and tuning
  - Or luck ☺

AlphaGo

AlphaGo – AI trained to play Go

Go = Game like chess with many more possible moves

- Go: 19x19 board ($\approx 10^{170}$ states) vs Chess: 8x8 board ($\approx 10^{47}$ states)
- Go: ≈ 200-300 possible moves/turn vs Chess: ≈ 35 possible moves/turn
- Go: ≈ 200-300 moves/game vs Chess: ≈ 40-60 moves/game

Made waves in 2016 by soundly beating top human players

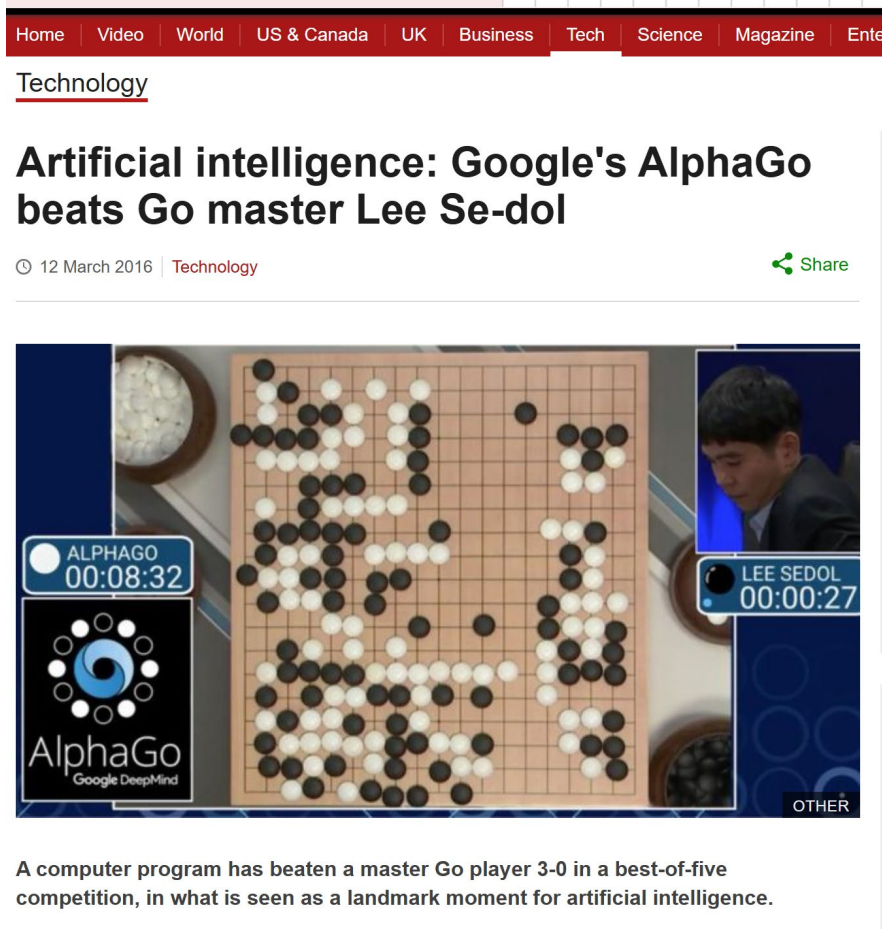
- Similar to chess breakthrough in 1997 but thought to be much harder

"We created AlphaGo, an AI system that combines deep neural networks with advanced search algorithms.

One neural network — known as the 'policy network' — selects the next move to play. The other neural network — the 'value network' — predicts the winner of the game.

Initially, we introduced AlphaGo to numerous amateur games of Go so the system could learn how humans play the game. Then we instructed AlphaGo to play against different versions of itself thousands of times, each time learning from its mistakes — a method known as reinforcement learning. Over time, AlphaGo improved and became a better player."
https://deepmind.google/research/breakthroughs/alphago/ (accessed 2/14/25)

"Artificial intelligence: Google's AlphaGo beats Go master Lee Se-dol". BBC News. 12 March 2016. Archived



AlphaGo

AlphaGo project formed around 2014

Estimated training cost (from https://klu.ai/glossary/alphago):

- $35M energy
- $25M hardware (trained on Google TPUs – specialized processors)
- ?? Human capital
- Not sure I buy these numbers and would certainly be much cheaper now
  - Hardware is cheaper and better
  - Algorithms are better
  - AI engineers have better understanding
    - AlphaGo played millions of games and trained for months, AlphaGo Zero (2017) played 4.9 million games and trained for 40 days, AlphaZero (2017) played 21 million games and trained for 3 days

Building big RL systems can be expensive

Dynamic Pricing

Fixed resource (e.g. hotel rooms, airline seats), time-varying customer demand, ability to pre-book

- Agent: pricing algorithm
- Actions: set of allowed prices/price adjustments
  - What might one think about here?
- Environment: marketplace
- State space: market conditions, customer type
  - current price, competitor prices, time of day, seasonality, demand conditions, supply conditions, business/leisure, …
- Reward
  - Revenue?
  - Customer satisfaction?
  - What can be measured?

Implementation in the real world

- How long to accumulate enough data?
- What happens during training period? Monitoring and guardrails?
- “Stationarity”?
- Noise?

5. Summary

Summary

Reinforcement learning is computerized learning by doing

Bandits leading example that is easy to understand and use

RL has lots of potential applications

- Be wary of overclaiming simplicity/scaleability though

Important current use of RL is RLHF

- RLHF = Reinforcement learning with human feedback
- Input into training LLMs